

Project Readme Template

Version 1 9/11/24

A single copy of this template should be filled out and submitted with each project submission, regardless of the number of students on the team. It should have the name `readme_”teamname”`. Also change the title of this template to “Project x Readme Team xxx”

1	Team Name: Margaret Aiyenero										
2	Team members names and netids Margaret Aiyenero : maiyener										
3	Overall project attempted, with sub-projects: Tracing NTM Behavior										
4	Overall success of the project: It runs and is completed										
5	Approximately total time (in hours) to complete: 7										
6	Link to github repository: https://github.com/maiyenero/Project2-TOC										
7	List of included files (if you have many files of a certain type, such as test files of different sizes, list just the folder): (Add more rows as necessary). Add more rows as necessary. <table border="1"><thead><tr><th>File/folder Name</th><th>File Contents and Use</th></tr></thead><tbody><tr><td colspan="2">Code Files</td></tr><tr><td>code.py</td><td>Main code file where we parse a csv file and extract the formal description of a non deterministic Turing machine and a function that simulates a non deterministic Turing machine</td></tr><tr><td colspan="2">Test Files</td></tr><tr><td> 1. a_plus_accept.csv 2. aa_plus_bb_accept_1.csv 3. aa_plus_bb_accept_2.csv 4. aa_plus_bb_reject.csv 5. one_plus_zero_accept_1.csv 6. one_plus_zero_accept_2.csv 7. one_plus_zero_reject.csv </td><td>The files are in the input folders. I edited the original one slightly and then added my own that work with accept and reject strings. There are 3 different machines with different strings.</td></tr></tbody></table>	File/folder Name	File Contents and Use	Code Files		code.py	Main code file where we parse a csv file and extract the formal description of a non deterministic Turing machine and a function that simulates a non deterministic Turing machine	Test Files		1. a_plus_accept.csv 2. aa_plus_bb_accept_1.csv 3. aa_plus_bb_accept_2.csv 4. aa_plus_bb_reject.csv 5. one_plus_zero_accept_1.csv 6. one_plus_zero_accept_2.csv 7. one_plus_zero_reject.csv	The files are in the input folders. I edited the original one slightly and then added my own that work with accept and reject strings. There are 3 different machines with different strings.
File/folder Name	File Contents and Use										
Code Files											
code.py	Main code file where we parse a csv file and extract the formal description of a non deterministic Turing machine and a function that simulates a non deterministic Turing machine										
Test Files											
1. a_plus_accept.csv 2. aa_plus_bb_accept_1.csv 3. aa_plus_bb_accept_2.csv 4. aa_plus_bb_reject.csv 5. one_plus_zero_accept_1.csv 6. one_plus_zero_accept_2.csv 7. one_plus_zero_reject.csv	The files are in the input folders. I edited the original one slightly and then added my own that work with accept and reject strings. There are 3 different machines with different strings.										

		- The file contents are listed in this order: - Name of machine - String - Q - Characters from E - Characters from Gamma - Start state - Accept state - Reject state
	Output Files	
	- a_plus_accept.pdf - aa_plus_bb_accept_1.pdf - aa_plus_bb_accept_2.pdf - aa_plus_bb_reject.pdf - one_plus_zero_accept_1.pdf - one_plus_zero_accept_2.pdf - one_plus_zero_reject.pdf	The output files show the name of the machines used, the string processed, configurations at each depth, level of non determinisitc and the transition log
	Plots (as needed)	
	table.png	Located in the plot section on github - Name - Input string - Result (accept/reject) - Depth #of configurations explored Average non determinism
8	Programming languages used, and associated libraries: python, csv, collections	
9	Key data structures (for each sub-project): tree and dictionary	
10	General operation of code (for each subproject): This code simulates a nondeterministic Turing Machine (NTM). The csv files are parsed by the code.py function and it reads in the machine's configurations, states, transitions, and the input string. The simulate_ntm	

	function explores all possible configurations of the machine, using BFS to process the transitions. It keeps track of the total number of accepted and rejected configurations as well as the level of nondeterminism until it either accepts, rejects or reaches a depth limit. The program finally outputs a detailed transcript of configurations and transitions during execution.
11	What test cases you used/added, why you used them, what did they tell you about the correctness of your code. 1. a_plus_accept.csv - input: aaa - tests a simple machine that matches the a^* pattern - the machine accepts this string, and it identifies all configurations demonstrating its ability to handle single character repetition 2. aa_plus_bb_accept_1.csv - input: aaabb - tests machine ability to handle concatenated patterns $(a+b)^*(aa+bb)(a+b)^*$ - the machine accepts this string, and it identifies all configurations demonstrating its ability to handle nested patterns 3. aa_plus_bb_accept_2.csv - input: aabb - tests the machines ability to process variations of the pattern $(a+b)^*(aa+bb)(a+b)^*$ without trailing characters - the machine accepts this string and it identifies all configurations demonstrating its ability to handle flexibility in the middle substring pattern 4. aa_plus_bb_reject.csv - input: abab - confirms that the machine rejects invalid patters that don't conform to $(a+b)^*(aa+bb)(a+b)^*$ - the machine correctly rejects the input string and proves that it can decipher between valid and invalid strings 5. one_plus_zero_accept_1.csv - input: 10010 - tests the machines ability to handle more complex binary patterns $1(0+1)^*0$ - the machines correctly accepts the string confirming that it can process binary strings with a valid prefix and suffix 6. one_plus_zero_accept_2.csv - input: 10 - tests the machines ability to verify shorter binary strings - the machines correctly accepts the input and proves that it can handle small valid cases 7. one_plus_zero_reject.csv - input: 10011 - tests the machines ability to reject invalid binary strings by $1(0+1)^*0$ - the machines rejects this string which confirms that it can identify invalid matches
12	How you managed the code development I used git commands and git hub. I tested the functions I made before moving on and

	then debugged after to make sure everything ran smoothly with the directions.
13	Detailed discussion of results: The simulator traced all possible paths for a nondeterministic Turing machine (NTM) using a BFS. This made sure that everything was correct and prevented infinite loops Accepting and rejecting states were identified in all test inputs BFS checked for all path exploration and captured all configurations to ensure acceptances were not missed Depth, configurations and average nondeterminism reflected the complexity of the machines and verified the implantation alignment with theoretical behavior I set the max depth limit to 20 to balance completeness and performance. BFS allows for high usage, so I needed to limit it to an appropriate amount decrease resource usage The results validated the simulators correctness which was the point of the entire project, so I was able to meet the goals :)
14	How team was organized : it was just me
15	What you might do differently if you did the project again: I would probably use a class to organize the attributes better to improve readability. I would also separate the parsing, simulation and logging into components and add type annotations for clarity and handling.
16	Any additional material: none